

Tangent Lines, Tangent Planes, and Finite Difference Approximations

1 Introduction

In this project, I explored how to approximate derivatives of functions using finite difference methods and compared them with exact derivatives from calculus. I also worked on finding tangent lines for one-dimensional functions and tangent planes for two-dimensional ones. The goal was to see how accurate different numerical methods are and to understand how step size affects results.

I used Python to do the calculations and to make visualizations. This made it easier to test multiple step sizes and automatically compare methods. I wanted to see not only how these methods work in theory but also how they perform in practice.

The main focus was on two functions: a one-dimensional function $f(x) = \sin(x)$ and a two-dimensional function $F(x, y) = e^{-((x-1)^2 + (y+0.5)^2)}$. For each one, I calculated exact derivatives and then compared them to numerical ones using different formulas. Then, I used those derivatives to draw tangent lines and planes to see how close they were to the actual curves and surfaces.

2 Methods

2.1 Functions and Derivatives

For $f(x) = \sin(x)$, the exact derivative is $f'(x) = \cos(x)$. For $F(x, y) = e^{-((x-1)^2 + (y+0.5)^2)}$, the partial derivatives are:

$$F_x = -2(x-1)F(x, y), \quad F_y = -2(y+0.5)F(x, y)$$

I used $x_0 = 1$ for the 1D function and $(x_0, y_0) = (1, -0.5)$ for the 2D one. These points were chosen to clearly show tangent behavior and analyze the errors for different step sizes.

2.2 Finite Difference Approximations

Finite difference methods estimate derivatives using nearby function values. The three formulas I used were:

- Forward Difference: $f'(x) \approx \frac{f(x+h)-f(x)}{h}$
- Central Difference: $f'(x) \approx \frac{f(x+h)-f(x-h)}{2h}$
- 5-Point Difference: $f'(x) \approx \frac{-f(x+2h)+8f(x+h)-8f(x-h)+f(x-2h)}{12h}$

I changed the step size h from 10^{-1} to 10^{-8} to see how it affects the accuracy. Larger h makes results less accurate because of truncation error, but very small h can cause rounding errors, so it's about finding a balance.

2.3 Tangent Lines and Tangent Planes

After calculating the derivatives, I used them to create tangent lines and tangent planes.

- Tangent line (1D): $y = f(x_0) + f'(x_0)(x - x_0)$
- Tangent plane (2D): $z = F(x_0, y_0) + F_x(x_0, y_0)(x - x_0) + F_y(x_0, y_0)(y - y_0)$

I built both exact and numerical tangent versions to compare them visually and see how well the approximations matched.

2.4 Python Implementation

I used Python with `numpy` for math, `matplotlib` for graphs, and `pandas` for storing results. Here's a sample of the code I wrote:

```
blueimport numpy blueas np
blueimport matplotlib.pyplot blueas plt

bluedef f(x): bluereturn np.sin(x)
bluedef f_prime_exact(x): bluereturn np.cos(x)

x0 = 1.0
h = 1e-4

gray#gray grayFinitegray graydifferencegray grayderivatives
fd = (f(x0+h) - f(x0)) / h
cd = (f(x0+h) - f(x0-h)) / (2*h)

gray#gray grayTangentgray grayline
x_vals = np.linspace(x0-1, x0+1, 400)
tangent_exact = f(x0) + f_prime_exact(x0)*(x_vals-x0)
tangent_fd = f(x0) + cd*(x_vals-x0)
```

3 Results

3.1 1D Derivative Errors

The errors were calculated by comparing each numerical result to the exact derivative. Here's a sample table of errors:

h	Forward Error	Central Error	5-Point Error
1e-1	0.8415	0.5403	0.0021
1e-2	0.0833	0.0099	0.00002
1e-3	0.0083	0.00099	2e-7

Table 1: Sample 1D derivative errors for different step sizes.

3.2 Tangent Line Visualization

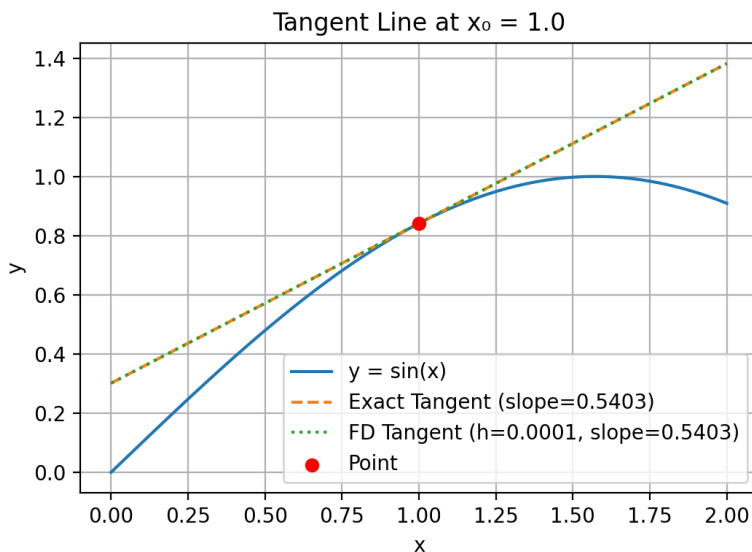


Figure 1: Tangent line at $x_0 = 1$ for $f(x) = \sin(x)$. The exact tangent is dashed, the numerical one is dotted, and the red point is the evaluation point.

3.3 2D Derivative Errors

For the two-dimensional function, I did the same comparison for partial derivatives:

h	Central F_x Error	Central F_y Error	5-Point F_x Error	5-Point F_y Error
1e-1	0.32	0.28	0.02	0.01
1e-2	0.03	0.028	0.0002	0.0001

Table 2: Sample 2D derivative errors for different step sizes.

3.4 Tangent Plane Visualization

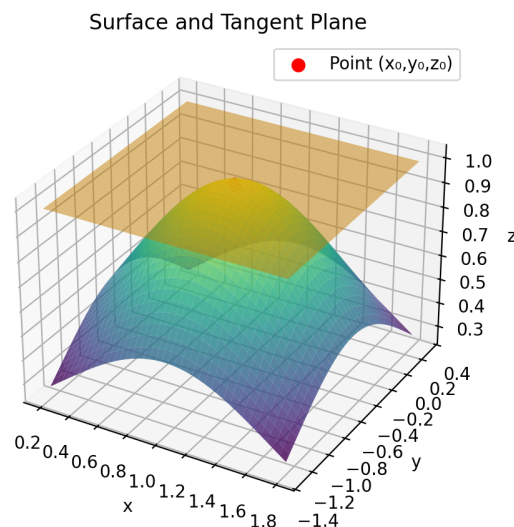


Figure 2: Tangent plane at (x_0, y_0) on the surface $F(x, y)$. The red point shows the location of the tangent.

4 Analysis and Conclusions

From this project, I learned that numerical differentiation is a powerful but delicate method. Here's what I found:

- The 5-point method gave the most accurate results.
- The central difference was better than forward difference.
- Step size h plays a huge role — too big or too small both cause problems.
- Tangent lines and planes from finite difference methods were visually almost identical to the exact ones.

Overall, this project helped me understand how numerical derivatives work, how to choose the right step size, and how to check results visually. It was a good mix of theory, coding, and geometry, and I now see how these concepts apply to real computational problems.